Contents

1 transform.cpp **1**

1.1 Transform::AddChild(Transform* tr) . . . 1
1.2 Transform::AddComponent(Component* component) . . . 2
1.3 Transform::FindChildByName(std::string name) . . . 3
1.4 Transform::FindComponentByUID(uint64$_t$ id) . . . 3
1.5 Transform::FindTransformByUID(uint64$_t$ id) . . . 3
1.6 Transform::GetChildren() . . . 4
1.7 Transform::GetComponent(std::string name) . . . 4
1.8 Transform::GetGlobalPosition() . . . 4
1.9 Transform::GetGlobalRotation() . . . 4
1.10 Transform::GetGlobalScale() . . . 4
1.11 Transform::GetLocalPosition() . . . 4
1.12 Transform::GetLocalRotation() . . . 4
1.13 Transform::GetLocalScale() . . . 5
1.14 Transform::Rotate(glm::vec3 axis, float amount) . . . 5
1.15 Transform::Scale(glm::vec3 amount) . . . 5
1.16 Transform::Translate(glm::vec3 offset) . . . 6
1.17 Transform::Forward() . . . 6
1.18 Transform::Backward() . . . 6
1.19 Transform::Left() . . . 6
1.20 Transform::Right() . . . 6
1.21 Transform::Up() . . . 7
1.22 Transform::Down() . . . 7

1 transform.cpp

1.1 Transform::AddChild(Transform* tr)

Specification

```
/**
 * Add a child transform to this transform
 *
 * If the desired child is already a child of this transform
 * log a warning and ignore it
 *
 * @param tr the transform to add as a child
 *
 */
void AddChild(Transform *tr);
```

Equivalence Classes

Eq. Class ID	Param	Description	Representative
1.1.EC1.1	tr	a valid pointer	new Transform()
1.1.EC1.a	tr	a null pointer	nullptr

Test Cases

Test Case ID	Param. *tr*	Expected
1.1.TC1	A pointer to some valid Transform	No output
1.1.TC1a	nullptr	Error: log message

Test Implementation

```
TEST_CASE("1.1.TC1") {

  Transform* test_transform = new Transform();
  Transform* child_transform = new Transform();

  //redirect cout to a string so we can check for the error message
  std::stringstream buffer;
  std::streambuf* old_cout = std::cout.rdbuf(buffer.rdbuf());

  test_transform->AddChild(child_transform);

  std::cout.rdbuf(old_cout);
  CHECK(buffer.str() == ""); //no output expected

  delete child_transform;
  delete test_transform;
}

TEST_CASE("1.1.TC2") {

  Transform* test_transform = new Transform();

  std::stringstream buffer;
  std::streambuf* old_cout = std::cout.rdbuf(buffer.rdbuf());

  test_transform->AddChild(nullptr);

  std::cout.rdbuf(old_cout);
  CHECK(buffer.str() != ""); //some kind of log output expected

  delete test_transform;
}
```

1.2 Transform::AddComponent(Component* component)

Equivalence Classes

Eq. Class ID	Param	Description	Representative
1.2.EC1.1	component	a valid pointer	new Component()
1.2.EC1.a	component	an invalid pointer	nullptr

Test Cases

Test Case ID	Param. *tr*	Expected
1.1.TC1	A pointer to some valid Transform	No output
1.1.TC1a	nullptr	Error: log message and ignore

1.3 Transform::FindChildByName(std::string name)

Equivalence Classes

Eq. Class ID	Param	Description	Representative
1.3.EC1.1	name	A string	"string"
1.3.EC1.a?	name	The empty string	""
1.3.EC1.b	name	Not a string	1

Test Cases

Test Case ID	Param. *name*	Expected
1.1.TC1.1	"string"	nullptr
1.1.TC1.a?	""	???
1.1.TC1.b	1	This will be disallowed by type checking, so TC implementation is omitted.

- The specification is unclear on whether the empty string is a valid name for a Transform

1.4 Transform::FindComponentByUID(uint64$_t$ id)

Equivalence Classes

Eq. Class ID	Param	Description	Representative
1.4.EC1.1	id	A positive integer	1
1.4.EC1.2	id	Zero	0
1.4.EC1.a	id	A negative integer	-1

Test Cases

Test Case ID	Param. *tr*	Expected	Coverage
1.4.TC1.1	1	nullptr	1.4.EC1.1
1.4.TC1.2	0	nullptr	1.4.EC1.2
1.4.TC1.a	-1	Disallowed by type checking, TC implementation omitted	1.4.EC1.a

- 1.1.TC1.a omitted, as it would be disallowed by the type checker.

1.5 Transform::FindTransformByUID(uint64$_t$ id)

Equivalence Classes

Equivalence classes same as Transform::FindComponentByUID.

Test Cases

Test Case ID	Param. *tr*	Expected	Coverage
1.5.TC1.1	1	nullptr	1.5.EC1.1
1.5.TC1.2	0	nullptr	1.5.EC1.2
1.5.TC1.a	-1	Disallowed by type checking, TC implementation omitted	1.5.EC1.a

1.6 Transform::GetChildren()

Equivalence Classes

Returns an instance variable that will always be valid.

1.7 Transform::GetComponent(std::string name)

Equivalence Classes

Possible issues with specification:

- The method to get a transform by name is called 'FindChildByName', this method should be updated to match that scheme as 'FindComponentByName'.

  The equivalence classes are the same as Transform::FindChildByName.

1.8 Transform::GetGlobalPosition()

Equivalence Classes

Eq. Class ID	Param	Description	Representative
1.8.EC1.1	parent (implicit)	parent is an invalid pointer	nullptr
1.8.EC1.2	parent (implicit)	parent is a valid pointer	new Transform()

1.9 Transform::GetGlobalRotation()

Equivalence Classes

Equivalence classes same as Transform::GetGlobalPosition.

1.10 Transform::GetGlobalScale()

Equivalence Classes

Equivalence classes same as Transform::GetGlobalScale.

1.11 Transform::GetLocalPosition()

Equivalence Classes

This is a simple getter and only exposes a field.

1.12 Transform::GetLocalRotation()

Equivalence Classes

This is a simple getter and only exposes a field.

1.13 Transform::GetLocalScale()

Equivalence Classes

This is a simple getter and only exposes a field.

1.14 Transform::Rotate(glm::vec3 axis, float amount)

Equivalence Classes

Eq. Class ID	Param	Description	Representative
1.14.EC1.1	axis	axis is a vector with nonzero length	{1.0,0.0,0.0}
1.14.EC1.a	axis	axis is a vector with zero length	{0.0,0.0,0.0}
1.14.EC1.b	axis	axis is not a vector	1
1.14.EC2.1	amount	amount is a float	90.0f
1.14.EC2.2	amount	amount is an integer	90
1.14.EC2.a	amount	amount is not a number	"90"

Test Cases

Test Case ID	Param.		Expected	Additional Coverage
	axis	*amount*		
1.14.TC1	{1.0,0.0,0.0}	90.0f	No output	1.14.EC1.1 1.14.EC2.1
1.14.TC2	{1.0,0.0,0.0}	90	No output	1.14.EC2.2
1.14.TC3	{0.0,0.0,0.0}	90.0f	Error: log message	1.14.EC1.a
1.14.TC4*	1	90.0f	Disallowed by type checker	1.14.EC1.b
1.14.TC5*	{1.0,0.0,0.0}	"90"	Disallowed by type checker	1.14.EC2.a

1.15 Transform::Scale(glm::vec3 amount)

Equivalence Classes

Eq. Class ID	Param	Description	Representative
1.15.EC1.1	amount	Any vector other than the zero vector or the identity vector of component-wise multiplication	{x=2.0,y=3.0,z=2.0}
1.15.EC1.2	amount	The identity element of component-wise multiplication	{x=1.0,y=1.0,z=1.0}
1.15.EC1.3	amount	The zero vector	{x=0.0,y=0.0,z=0.0}
1.15.EC1.a	amount	Not a vector	1

Test Cases

Test Case ID	Param. *amount*	Expected	Coverage
1.15.TC1	{2.0,3.0,2.0}	No output	1.15.EC1.1
1.15.TC2	{1.0,1.0,1.0}	No output	1.15.EC1.2
1.15.TC3	{0.0,0.0,0.0}	No output	1.15.EC1.3
1.15.TC4*	1	Disallowed by type checker	1.15.EC1.a

1.16 Transform::Translate(glm::vec3 offset)

Equivalence Classes

Eq. Class ID	Param	Description	Representative
1.16.EC1.1	amount	Any vector that is not the identity vector	{x=2.0,y=3.0,z=2.0}
1.16.EC1.2	amount	The identity vector for addition	{x=0.0,y=0.0,z=0.0}
1.16.EC1.a	amount	Not a vector	1

Test Cases

Test Case ID	Param. *amount*	Expected	Coverage
1.16.TC1	{2.0,3.0,2.0}	No output	1.16.EC1.1
1.16.TC2	{0.0,0.0,0.0}	No output	1.16.EC1.2
1.16.TC3	1	Disallowed by type checker	1.16.EC1.a

1.17 Transform::Forward()

Equivalence Classes

Method has no parameters.

1.18 Transform::Backward()

Equivalence Classes

Method has no parameters.

1.19 Transform::Left()

Equivalence Classes

Method has no parameters.

1.20 Transform::Right()

Equivalence Classes

Method has no parameters.

1.21 Transform::Up()

Equivalence Classes

Method has no parameters.

1.22 Transform::Down()

Equivalence Classes

Method has no parameters.